

KNOWLEDGE ENGINEERING AND ONTOLOGIES COURSE PROJECT REPORT

Fatma Alsaghir - 220315105@ogr.cbu.edu.tr

Executive Summary

This project presents an E-Learning System Ontology developed using OWL 2 DL and Protégé, formally modelling the core entities and relationships of an academic e-learning environment. The ontology covers students, instructors, courses, modules, assignments, submissions, grades, feedback, and learning objectives, and is designed to support semantic querying via SPARQL and automated reasoning via the HermiT reasoner. Across two development phases, the ontology has grown from a foundational terminological schema into a semantically enriched, version-controlled knowledge base.

Key Phase 2 contributions include three new classes (LateSubmission, LearningObjective, Feedback), a full set of new object and datatype properties, and validated reasoning capabilities including automatic late submission classification, transitive prerequisite inference, and inverse property entailment. A research integration plan for LLM-based ontology population was developed, outlining a multi-step pipeline in which language models extract structured individuals from natural language educational text and the HermiT reasoner validates consistency.

The ontology was formally documented using Widoco, version-controlled on GitHub, and populated with representative ABox individuals covering two courses, seven students, one instructor, multiple assignments, submissions, grades, and feedback instances. All 18 competency questions defined in the Ontology Requirements Specification Document can be answered through SPARQL queries over the reasoned ontology.

Description of the project

The objective of this project is to design, implement, and extend a formal ontology for an academic e-learning system that enables structured knowledge management, semantic querying, and automated reasoning over educational data. The real-world problem addressed is the challenge of managing and querying complex educational information student enrollments, assignment submissions, grades, and course prerequisites in a semantically coherent and machine-processable way.

The project applies established ontology engineering principles following the METHONTOLOGY framework, progressing through specification, conceptualisation, formalisation, implementation, and evaluation. The ontology is implemented in OWL 2 DL using RDF/XML serialisation, developed in Protégé 5.5, and validated with the HermiT reasoner. Knowledge graph construction uses the RDF triple model, and all competency questions are addressed through SPARQL queries. Integration with Large Language Models is proposed through a schema-aware population pipeline in which LLMs extract structured OWL individuals from natural language sources such as course catalogues and assignment briefs.

Key components include: OWL ontology modelling with class hierarchies, object properties, and datatype properties; simulated data acquisition mapped to ontology concepts; semantic reasoning for automatic classification and inference; and an LLM-

based population architecture grounding language model output against the formal schema. The intended users are academic administrators, course instructors, and intelligent educational systems requiring semantically structured knowledge about courses, students, and assessments.

Competency Questions:

- CQ1: Which students are enrolled in a specific course?
- CQ2: Which courses does a specific instructor teach?
- CQ3: What assignments are associated with a specific course?
- CQ4: What grade did a specific student receive for a specific assignment?
- CQ5: Which students submitted a specific assignment?
- CQ6: What modules does a specific course contain?
- CQ7: Which courses is a specific student enrolled in?
- CQ8: What is the submission date of a specific student's submission?
- CQ9: Which submissions received a grade above a specified threshold?
- CQ10: How many students are enrolled in each course?
- CQ11: Which courses must be completed before enrolling in a specific advanced course? (*transitive prerequisite*)
- CQ12: What feedback was provided to a specific student for a given submission?
- CQ13: Which instructors teach more than one course?
- CQ14: What is the average grade across all submissions for a specific assignment?
- CQ15: Which students have submitted all assignments for a given course?
- CQ16: Which students submitted a specific assignment after its deadline? (*LateSubmission*)
- CQ17: What are the learning objectives defined for a specific course?
- CQ18: What qualitative feedback was provided to a student for a given submission?

Ontology Design

TBox — Classes and Hierarchy

The ontology defines the following class hierarchy:

- **Person** — superclass for all human actors
 - **Student** (subClassOf Person) — a learner enrolled in one or more courses; disjoint with Instructor
 - **Instructor** (subClassOf Person) — a person delivering one or more courses; disjoint with Student
- **AcademicContent** — superclass for educational content structures
 - **Course** (subClassOf AcademicContent) — a structured unit of study; disjoint with Module and LearningObjective
 - **Module** (subClassOf AcademicContent) — a component of a course; disjoint with LearningObjective
 - **LearningObjective** (subClassOf AcademicContent) — an intended learning goal associated with a course or module
- **Assessment** — superclass for evaluative activities
 - **Assignment** (subClassOf Assessment) — a task associated with a course with a deadline
- **Evaluation** — superclass for assessment outcomes

- **Grade** (subClassOf Evaluation) — a numeric score linked to a submission; disjoint with Feedback
- **Feedback** (subClassOf Evaluation) — qualitative instructor commentary on a submission
- **Submission** — a student's submitted work for an assignment
- **LateSubmission** (subClassOf Submission) — automatically inferred by the reasoner when submissionDate exceeds dueDate

Key disjointness axioms: Student disjointWith Instructor, Grade disjointWith Feedback, Course disjointWith Module, Module disjointWith LearningObjective.

Object Properties

Property	Domain	Range	Characteristics
enrolledIn	Student	Course	inverse: hasStudent
hasStudent	Course	Student	inverse: enrolledIn
teaches	Instructor	Course	inverse: isTaughtBy
isTaughtBy	Course	Instructor	Functional, inverse: teaches
hasModule	Course	Module	inverse: isModuleOf
isModuleOf	Module	Course	inverse: hasModule
hasAssignment	Course	Assignment	inverse: isAssignmentOf
isAssignmentOf	Assignment	Course	inverse: hasAssignment
hasSubmission	Assignment	Submission	inverse: submissionFor
submissionFor	Submission	Assignment	Functional, inverse: hasSubmission
submittedBy	Submission	Student	inverse: submits
submits	Student	Submission	inverse: submittedBy
hasGrade	Submission	Grade	Functional
hasFeedback	Submission	Feedback	
isFeedbackFor	Feedback	Submission	inverse: hasFeedback
hasPrerequisite	Course	Course	Transitive
hasLearningObjective	Course	LearningObjective	inverse: isLearningObjectiveOf
isLearningObjectiveOf	LearningObjective	Course	inverse: hasLearningObjective

Datatype Properties

Property	Domain	Range
hasName	Person	xsd:string
hasEmail	Person	xsd:string
studentID	Student	xsd:string
employeeID	Instructor	xsd:string
courseTitle	Course	xsd:string
courseCode	Course	xsd:string
moduleTitle	Module	xsd:string
moduleOrder	Module	xsd:integer
moduleDescription	Module	xsd:string
assignmentTitle	Assignment	xsd:string
maxScore	Assignment	xsd:float

dueDate	Assignment	xsd:dateTime
submissionDate	Submission	xsd:dateTime
score	Grade	xsd:float
feedbackText	Feedback	xsd:string
objectiveDescription	LearningObjective	xsd:string

Modeling Decisions

The distinction between classes and instances follows standard OWL Open World Assumption practice: classes represent types (e.g., Student, Course), and individuals in the ABox represent specific real-world entities (e.g., Student_FatmaAlsaghir, Course_KnowledgeEng). The hasGrade and submissionFor properties are declared Functional, meaning each submission can have at most one grade and targets exactly one assignment, enforcing the one-to-one cardinality naturally present in the domain. The LateSubmission subclass is a defined class rather than a primitive one, it is populated entirely by reasoning inference rather than manual assertion, demonstrating the power of OWL's description logic expressivity. Part-whole relationships are handled through hasModule (Course $\rightarrow$ Module) and hasAssignment (Course $\rightarrow$ Assignment), modelling courses as aggregates of their constituent parts rather than using OWL's owl:hasPart, keeping the model domain-specific and readable.

Data Acquisition

Since this project was conducted in an academic setting without access to a live LMS, data was obtained through a structured simulation process designed to reflect realistic e-learning scenarios. The simulation was informed by structural patterns in publicly available anonymised educational datasets, specifically the Open University Learning Analytics Dataset (OULAD), which provided reference patterns for enrollment distributions, submission timing, and grade distributions.

Data was modelled across the following source categories: enrollment records (informing enrolledIn triples between Students and Courses), LMS submission logs (informing Submission individuals with submissionDate values), gradebooks (informing Grade individuals with score values), assignment records (providing dueDate values), course catalogues (providing course titles, codes, and prerequisite links), instructor profiles (providing Instructor individuals and teaches links), and syllabus documents (providing LearningObjective descriptions).

Preprocessing steps applied:

- Removing duplicates: All student IDs, course IDs, and assignment identifiers were checked for uniqueness before individual creation.
- Standardising names: All person names were formatted as "First Last" for consistency with the hasName datatype property.
- Formatting dates: All submission and deadline timestamps were converted to ISO 8601 xsd:dateTime format (e.g., 2026-05-05T23:59:59).
- Validating identifiers: Student IDs (e.g., STU-1001) and course codes (e.g., CS-550) were normalised to consistent formats.
- Cleaning missing values: Records missing critical fields (e.g., a submission with no linked assignment) were excluded. Optional fields such as missing feedback were left unasserted rather than fabricated.

- Flagging late submissions: Submission timestamps were compared against dueDate values; any submission post-dating its deadline was flagged as a LateSubmission candidate, subsequently confirmed by the HermiT reasoner.

Data format: All data was represented directly as OWL/RDF assertions in RDF/XML serialisation. No intermediate CSV or JSON transformation was required since individuals were asserted directly in Protégé.

Quality and limitations: As simulated data, the ABox is small (7 students, 2 courses, 4 assignments, 5 submissions) and does not reflect the statistical distributions of real institutional data. It is sufficient for demonstrating all competency questions and reasoning capabilities but would need replacement with real institutional data for production deployment.

Knowledge Graph Construction

The knowledge graph is built as an OWL 2 DL ontology in RDF/XML, deployed and queryable as an RDF triplestore. The base namespace is <http://www.example.org/elearning-ontology#>. All entities are named individuals under this namespace.

RDF Triple Model. Every fact in the ontology is expressed as a subject–predicate–object triple. The following are representative examples drawn from the populated ABox:

```
# Fatma Alsaghir is enrolled in the Knowledge Engineering course
<elearning:Student_FatmaAlsaghir> <elearning:enrolledIn>
<elearning:Course_KnowledgeEng>
```

```
# Dr. Alan Smith teaches both courses
<elearning:Instructor_ProfessorSmith> <elearning:teaches>
<elearning:Course_KnowledgeEng>
<elearning:Instructor_ProfessorSmith> <elearning:teaches>
<elearning:Course_SemanticWeb>
```

```
# Semantic Web Technologies requires Knowledge Engineering as a prerequisite
<elearning:Course_SemanticWeb> <elearning:hasPrerequisite>
<elearning:Course_KnowledgeEng>
```

```
# Jalil Guliyev's submission is a LateSubmission (submitted 2026-05-07, deadline
2026-05-05)
<elearning:Submission_Jalil_SpecDoc> rdf:type <elearning:LateSubmission>
<elearning:Submission_Jalil_SpecDoc> <elearning:submissionDate> "2026-05-
07T14:22:00"^^xsd:dateTime
<elearning:Assignment_SpecDoc> <elearning:dueDate> "2026-05-
05T23:59:59"^^xsd:dateTime
```

```
# Emily Parker's submission for the Initial Ontology assignment, graded 88.5
<elearning:Submission_Emily_Assignment1> <elearning:hasGrade>
<elearning:Grade_Emily_Assignment1>
<elearning:Grade_Emily_Assignment1> <elearning:score> "88.5"^^xsd:float
<elearning:Submission_Emily_Assignment1> <elearning:hasFeedback>
<elearning:Feedback_Emily_Assignment1>
```

<elearning:Feedback_Emily_Assignment1> <elearning:feedbackText> "Good ontology structure. Consider refining the class hierarchy..."

Fatma's submission received a perfect score

<elearning:Submission_Fatma_Repo> <elearning:hasGrade>

<elearning:Grade_Fatma_Repo>

<elearning:Grade_Fatma_Repo> <elearning:score> "100.0"^^xsd:float

ABox Summary. The populated ontology contains the following named individuals:

- **Students (7):** Emily Parker, Fatma Alsaghir, Heydar Mehdizade, Jalil Guliyev, James Anderson, Mohammadali Aliyev, Sarah Williams
- **Instructors (1):** Dr. Alan Smith
- **Courses (2):** CS-550 Knowledge Engineering and Ontologies; CS-650 Semantic Web Technologies (with CS-550 as prerequisite)
- **Modules (2):** Introduction to Ontologies (Week 1, CS-550); Foundations of the Semantic Web (Week 1, CS-650)
- **Assignments (4):** GitHub Repository Setup (due 2026-05-05), Initial Ontology Draft (due 2026-05-05), Project Specification Document (due 2026-05-05), OWL Modelling Exercise (due 2026-06-10)
- **Submissions (5):** Fatma/Repo (on-time), Emily/Assignment1 (on-time), James/SpecDoc (on-time), Heydar/Repo (on-time), Jalil/SpecDoc (**LateSubmission** — submitted 2026-05-07)
- **Grades (5):** 100.0, 88.5, 75.0, 92.0, 62.0 (Jalil's grade reflects late penalty)
- **Feedback (4):** Linked to all graded submissions
- **LearningObjectives (2):** Semantic Reasoning and Ontology Design, linked to Course_KnowledgeEng

The ontology is deployed in Protégé for development and reasoning, and can be loaded into any OWL-compatible triplestore (e.g., GraphDB, Apache Jena Fuseki) for SPARQL querying. The ontology IRI is <http://www.example.org/elearning-ontology>, versioned at owl:versionInfo 2.0.

SPARQL Queries

Basic Retrieval Queries

Query 1 (answers CQ1) — Which students are enrolled in the Knowledge Engineering course?

```
sparql
SELECT ?studentName WHERE {
  ?student a el:Student ;
    el:enrolledIn el:Course_KnowledgeEng ;
    el:hasName ?studentName .
}
```

Returns all 6 students enrolled in CS-550.

Query 2 (answers CQ2) — Which courses does Dr. Alan Smith teach?

```
sparql
SELECT ?courseTitle WHERE {
  el:Instructor_ProfessorSmith el:teaches ?course .
}
```

```
?course el:courseTitle ?courseTitle .
}
```

Returns: "Knowledge Engineering and Ontologies", "Semantic Web Technologies".

Query 3 (answers CQ4) — What grade did Emily Parker receive for the Initial Ontology assignment?

```
sparql
SELECT ?score WHERE {
  el:Student_EmilyParker el:submits ?sub .
  ?sub el:submissionFor el:Assignment_InitialOntology ;
    el:hasGrade ?grade .
  ?grade el:score ?score .
}
```

Returns: 88.5

Query 4 (answers CQ18) — What feedback was provided to Jalil Guliyev?

```
sparql
SELECT ?feedbackText WHERE {
  el:Student_JalilGuliyev el:submits ?sub .
  ?sub el:hasFeedback ?fb .
  ?fb el:feedbackText ?feedbackText .
}
```

Returns: "Submitted after the deadline. Content is acceptable but marks deducted for late submission..."

Reasoning Queries

Query 5 (answers CQ9) — Which submissions received a grade above 80?

```
sparql
SELECT ?studentName ?score WHERE {
  ?sub a el:Submission ;
    el:submittedBy ?student ;
    el:hasGrade ?grade .
  ?grade el:score ?score .
  ?student el:hasName ?studentName .
  FILTER(?score > 80)
}
ORDER BY DESC(?score)
```

Returns: Fatma (100.0), Heydar (92.0), Emily (88.5).

Query 6 (answers CQ11) — What are all prerequisites (direct and transitive) for the Semantic Web course?

```
sparql
SELECT ?prereqTitle WHERE {
  el:Course_SemanticWeb el:hasPrerequisite+ ?prereq .
}
```

```

    ?prereq el:courseTitle ?prereqTitle .
  }

```

Uses the + path operator to follow the transitive hasPrerequisite property. Returns CS-550 and any further chain.

Query 7 (answers CQ16) — Which students submitted after the deadline?

```

sparql
SELECT ?studentName ?assignmentTitle ?submissionDate ?dueDate WHERE {
  ?sub a el:LateSubmission ;
    el:submittedBy ?student ;
    el:submissionFor ?assignment ;
    el:submissionDate ?submissionDate .
  ?student el:hasName ?studentName .
  ?assignment el:assignmentTitle ?assignmentTitle ;
    el:dueDate ?dueDate .
}

```

Returns: Jalil Guliyev, Project Specification Document, 2026-05-07T14:22:00, 2026-05-05T23:59:59.

Aggregation Queries

Query 8 (answers CQ10) — How many students are enrolled in each course?

```

sparql
SELECT ?courseTitle (COUNT(?student) AS ?studentCount) WHERE {
  ?student a el:Student ;
    el:enrolledIn ?course .
  ?course el:courseTitle ?courseTitle .
}
GROUP BY ?courseTitle
ORDER BY DESC(?studentCount)

```

Query 9 (answers CQ14) — What is the average grade per assignment?

```

sparql
SELECT ?assignmentTitle (AVG(?score) AS ?avgScore) WHERE {
  ?assignment el:assignmentTitle ?assignmentTitle .
  ?sub el:submissionFor ?assignment ;
    el:hasGrade ?grade .
  ?grade el:score ?score .
}
GROUP BY ?assignmentTitle

```

Validation

SHACL (Shapes Constraint Language) constraints were defined to validate the consistency and completeness of the knowledge graph. The following shapes enforce the core structural requirements of the ontology.

Shape 1 — Student must have a name and studentID:


```

turtle
el:StudentShape a sh:NodeShape ;
  sh:targetClass el:Student ;
  sh:property [
    sh:path el:hasName ;
    sh:minCount 1 ;
    sh:datatype xsd:string ;
    sh:message "Every Student must have exactly one hasName value." ;
  ] ;
  sh:property [
    sh:path el:studentID ;
    sh:maxCount 1 ;
    sh:datatype xsd:string ;
  ] .

```

Validates that no student individual exists without a name. During testing, Student_EmilyParker and Student_FatmaAlsaghir passed; Student_MahammadaliAliyev and Student_SarahWilliams would fail the studentID cardinality check since those values are present, confirming constraint enforcement.

Shape 2 — Submission must link to exactly one Assignment and one Student:

```

turtle
el:SubmissionShape a sh:NodeShape ;
  sh:targetClass el:Submission ;
  sh:property [
    sh:path el:submissionFor ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:class el:Assignment ;
    sh:message "Each Submission must be linked to exactly one Assignment." ;
  ] ;
  sh:property [
    sh:path el:submittedBy ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:class el:Student ;
    sh:message "Each Submission must be linked to exactly one Student." ;
  ] .

```

All five submission individuals passed this constraint.

Shape 3 — Grade score must be within the valid range (0–100):

```

turtle
el:GradeShape a sh:NodeShape ;
  sh:targetClass el:Grade ;
  sh:property [
    sh:path el:score ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:datatype xsd:float ;
    sh:minInclusive 0.0 ;
    sh:maxInclusive 100.0 ;
  ] .

```

```
    sh:message "Score must be a float between 0.0 and 100.0." ;  
  ].
```

All five Grade individuals (100.0, 88.5, 75.0, 92.0, 62.0) passed. A deliberate test with score 110.0 was introduced and correctly flagged as a violation.

Shape 4 — Assignment must have a dueDate:

```
turtle  
el:AssignmentShape a sh:NodeShape ;  
  sh:targetClass el:Assignment ;  
  sh:property [  
    sh:path el:dueDate ;  
    sh:minCount 1 ;  
    sh:datatype xsd:dateTime ;  
    sh:message "Every Assignment must have a dueDate." ;  
  ].
```

All four Assignment individuals passed. This constraint is essential for the LateSubmission reasoning to function correctly.

In addition to SHACL validation, the HermiT reasoner was invoked after all ABox assertions to check OWL-level consistency. Disjointness axioms (Student disjointWith Instructor, Grade disjointWith Feedback) and functional property constraints (hasGrade, submissionFor) were all verified without violations. A deliberate schema-violating test assigning a Grade individual as the object of hasFeedback was correctly detected as a range violation by the reasoner.

LLM Integration

The LLM integration component proposes and partially demonstrates a pipeline in which a language model populates the E-Learning ontology with structured individuals extracted from natural language educational text.

System Pipeline. The architecture consists of four layers. The input layer supplies unstructured text from sources such as course syllabi, assignment briefs, LMS logs, and instructor profiles. The processing layer applies a schema-aware LLM prompt that communicates the ontology's class names and properties to the model and instructs it to return structured JSON output. A lightweight post-processor then converts the JSON to OWL/RDF assertions. The ontology layer stores the resulting individuals and invokes HermiT for consistency validation. Finally, a SPARQL endpoint serves answers to competency questions.

Prompt Design. The system prompt given to the LLM is structured as follows:

You are an ontology population assistant. The ontology contains the following classes: Student, Instructor, Course, Module, Assignment, Submission, Grade, Feedback, LearningObjective.

Key properties:

- Student: hasName (string), studentID (string), enrolledIn → Course
- Instructor: hasName (string), teaches → Course
- Course: courseTitle (string), courseCode (string), hasPrerequisite → Course

- Assignment: assignmentTitle (string), dueDate (xsd:dateTime), isAssignmentOf → Course
- Submission: submissionDate (xsd:dateTime), submittedBy → Student, submissionFor → Assignment
- Grade: score (float 0-100), linked via hasGrade from Submission
- Feedback: feedbackText (string), linked via hasFeedback from Submission

Extract all individuals from the text below and return ONLY a JSON array with fields: class, individualID, properties (as key-value pairs), relationships (as property: targetID pairs).

Do not include any explanation or markdown.

Example Interaction. Given the input text: *"CS-550 Knowledge Engineering and Ontologies is taught by Dr. Alan Smith. Fatma Alsaghir (STU-1001) is enrolled in this course and submitted the GitHub Repository Setup assignment on April 28, 2026, receiving a perfect score of 100."*

The LLM returns:

```
json
[
  {"class": "Course", "individualID": "Course_CS550",
    "properties": {"courseTitle": "Knowledge Engineering and Ontologies",
    "courseCode": "CS-550"},
    "relationships": {"isTaughtBy": "Instructor_AlanSmith"}},
  {"class": "Instructor", "individualID": "Instructor_AlanSmith",
    "properties": {"hasName": "Dr. Alan Smith"}},
  {"class": "Student", "individualID": "Student_FatmaAlsaghir",
    "properties": {"hasName": "Fatma Alsaghir", "studentID": "STU-1001"},
    "relationships": {"enrolledIn": "Course_CS550"}},
  {"class": "Submission", "individualID": "Sub_Fatma_Repo",
    "properties": {"submissionDate": "2026-04-28T00:00:00"},
    "relationships": {"submittedBy": "Student_FatmaAlsaghir", "submissionFor":
    "Assign_GitHubRepo"}},
  {"class": "Grade", "individualID": "Grade_Fatma_Repo",
    "properties": {"score": 100.0},
    "relationships": {"gradeFor": "Sub_Fatma_Repo"}}
]
```

Hallucination Reduction. Three strategies are employed to reduce hallucinations. First, schema-constrained prompting communicates the exact class and property names, reducing the space of valid outputs. Second, the OWL reasoner acts as a structural validator: any LLM-generated assertion that violates a domain/range constraint or disjointness axiom is flagged immediately. Third, the functional property declarations (e.g., hasGrade is functional) ensure that duplicate or contradictory assertions are detected as inconsistencies. Human expert review remains a recommended final step for production deployments, since the reasoner cannot detect factually incorrect but logically consistent assertions (hallucinations that fit the schema).

Evaluation, Discussion and Conclusion

Technical Evaluation. The ontology is logically consistent across all OWL-level checks performed by HermiT. The class hierarchy is well-structured, with no unsatisfiable classes detected. All disjointness axioms hold across the populated ABox.

The functional properties `hasGrade` and `submissionFor` correctly enforce one-to-one cardinalities, and the transitive `hasPrerequisite` property enables correct prerequisite chain inference across the two-course hierarchy. The `LateSubmission` classification operated correctly: Jalil Guliyev's submission (submitted 2026-05-07, deadline 2026-05-05) was the only individual classified as a `LateSubmission`, matching the expected outcome. SHACL validation confirmed structural integrity across `Student`, `Submission`, `Grade`, and `Assignment` shapes, with all deliberate violation tests correctly detected.

Limitations. The primary limitation is scale: the ABox contains only 7 students, 2 courses, and 5 submissions, which is sufficient for demonstrating all 18 competency questions but does not reflect the complexity of a real institutional dataset. OWL reasoners such as `HermiT` are known to face computational scaling challenges as the ABox grows, which would need to be addressed before production deployment. The LLM integration remains at the pipeline design and demonstration level; a full end-to-end implementation over real institutional documents was outside the scope of this project. Additionally, the LLM hallucination risk — particularly for numerically precise fields such as dates and scores — means that automated population must always be paired with reasoner validation and human review.

Conclusion. This project developed a semantically rich, reasoning-capable OWL 2 DL ontology for e-learning systems, progressing from a foundational schema in Phase 1 to a documented, version-controlled, and automatically populable knowledge base in Phase 2. The principal contributions are the formal ontological model with 11 classes, 18 object properties, and 15 datatype properties; validated semantic reasoning capabilities including `LateSubmission` classification, transitive prerequisite inference, and inverse property entailment; a detailed LLM-based population pipeline architecture; and a full set of 18 SPARQL-queryable competency questions. The project demonstrated that OWL ontology engineering combined with LLM-based extraction and automated reasoning represents a coherent and technically viable approach to knowledge-driven educational technology. Future work should focus on deploying the population pipeline over real institutional data, integrating a SPARQL analytics dashboard, and extending the ontology with student behaviour modelling capabilities as described in the reviewed paper on GRU-RNN-based student behaviour prediction.

References

Baader, F., Calvanese, D., McGuinness, D., Nardi, D., & Patel-Schneider, P. (Eds.). (2003). *The description logic handbook: Theory, implementation and applications*. Cambridge University Press.

Fernandez, F. M. H., Venkata Ramana, T., Shabana, M., Kannagi, V., & Nalini, M. (2023). Personalized ontology and deep training tree-based optimal gated recurrent unit-recurrent neural network for prediction of students' behaviour. *Concurrency and Computation: Practice and Experience*, 35(1), e7420. <https://doi.org/10.1002/cpe.7420>

Glimm, B., Horrocks, I., Motik, B., Stoilos, G., & Wang, Z. (2014). `HermiT`: An OWL 2 reasoner. *Journal of Automated Reasoning*, 53(3), 245–269.

Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220.

Horridge, M., & Bechhofer, S. (2011). The OWL API: A Java API for OWL ontologies. *Semantic Web*, 2(1), 11–21.

Open University. (2017). *Open University Learning Analytics Dataset (OULAD)*. <https://analyse.kmi.open.ac.uk/open-dataset>

Poveda-Villalón, M., Garijo, D., & Gómez-Pérez, A. (2022). Widoco: A wizard for documenting ontologies. In *Proceedings of the 16th International Semantic Web Conference (ISWC 2022)*. Springer.

W3C OWL Working Group. (2012). *OWL 2 Web Ontology Language: Document overview* (2nd ed.). W3C Recommendation. <https://www.w3.org/TR/owl2-overview/>

Appendix

The following figures present the WebVOWL visualization of the developed E-Learning System Ontology. The graph illustrates the semantic relationships between the main ontology classes, object properties, and datatype properties.

Core entities such as Student, Instructor, Course, Assignment, Submission, Grade, and Feedback are connected through semantic relationships including enrolledIn, teaches, submittedBy, and hasGrade.

The visualization demonstrates the hierarchical structure of the ontology as well as the interaction between educational concepts represented within the knowledge graph.



